Development of a Progressive Web App for Accomplishing SALN Forms.

Lucas Miguel V. Agcaoili, Miguel A. Guiang, Aeron Dann P. Peñaflorida

CS 192 TBC/HQR1 - LAMig

Instructor: Professor Ligaya Leah Figueroa

Client: Associate Professor Rommel Feria

Date of Submission: 2026-06-02

Website Link: https://lamig-saln.online/

Repository Link: https://github.com/SHIROKAMIQQ/SALN-App

I. Statement of Requirements

Background and Motivation

The SALN (Statements of Assets, Liabilities and Net Worth) is a required document that government officials must submit annually. It serves as a declaration of their assets (what they own), liabilities (what they owe), and total net worth, while also disclosing any financial connections or business interests. Given this, the SALN promotes transparency, accountability, and integrity in the public office, which serves as a safeguard against corruption. Currently, government officials still have to go through a tedious process of manually filling out forms either through handwriting or electronically using spreadsheets. Thus, there is a need for an application to make filling up SALNs more efficient.

Proposed Solution / Project Idea

The proposed solution is a SALN Progressive Web App that aims to streamline the generation of the SALN by allowing users to input their information through either completing a form digitally through the app or uploading a JSON. The app will then generate the input into a properly formatted PDF file. Since it is a progressive web app, it means that the web app must be easy to access, installable, available offline, background-capable, and responsive to any device on which it is installed. Additionally, the inputted information will only be temporarily stored, after which it will be deleted. This ensures user privacy when filling out sensitive information.

Users/Stakeholders

- **Practitioners** - The practitioners of this software project will be Migz, Lucas, and Dann, who are primarily responsible for carrying out the development tasks, software project paperwork, and communication with the client.
- **Customers** - The main customer of this software project is Prof. Rommel Feria, who will provide the project requirements, set expectations for deliverables, and give feedback to the project at every scheduled meeting.
- **End Users** - The end users for this project are government employees who need to file their SALN.

Requirements/Features

To register for an account, users will be asked to provide an email address. This will be used for Two-Factor Authentication during registration and for every login instance. After the user provides their email, they will be sent 4 random words as OTP, which they need to input to verify their account.

Once the user logs in, they will be taken to the dashboard with two buttons: one that brings them to the SALN page, and one that lets them view a saved SALN form that they filled out within the past 5 days.

Once the user goes to the SALN page, they can either fill out a form on the site or upload a JSON file with all the information they need to fill out the form. They can upload their signature to the site or sign it using their mouse or touchscreen. Upon completion of a SALN form, the user may generate a PDF of the completed form with the information they filled in. They then have the option to download or save the information they uploaded.

The app also allows the user to delete their account or remove any SALN form data. Lastly, SALN data older than 5 days will be automatically deleted from the database.

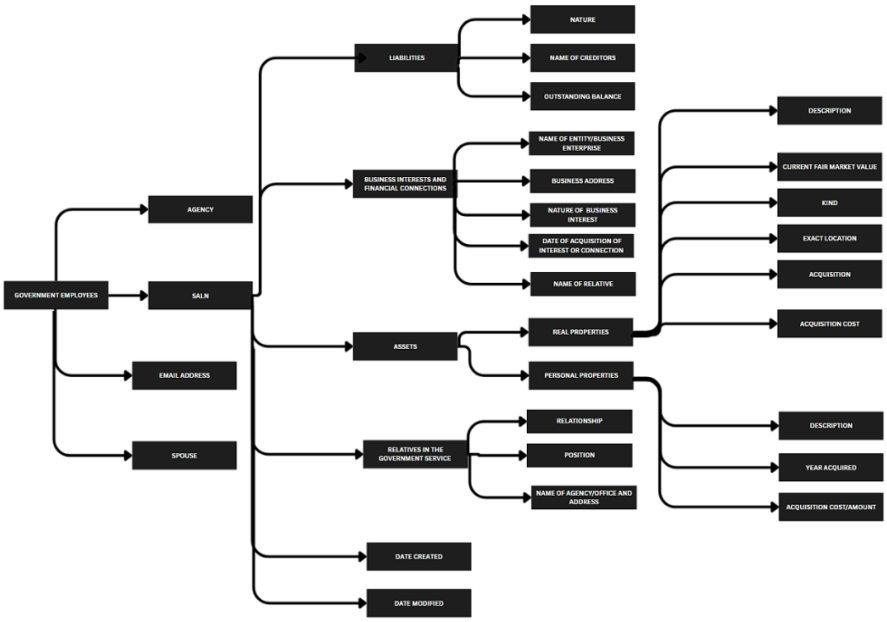
In essence, the app will be a progressive web application that allows users to create accounts to track the status of their SALN. Once the user requests to fill out their SALN, they will be able to access a digitalized interface similar to the SALN. There will be automatic computation for total assets and liabilities based on the inputs from the user, and a dropdown menu to allow them to add as many

family members or boxes as necessary. Information gathered by the app will be stored and encrypted in a secure database.

II. System Requirements Document

WEB APP REQUIREMENTS MODEL

Content Model



The core feature of the app is the SALN form, which government employees use to disclose their financial connections, assets (both real and personal), and corresponding liabilities. To maintain data security, metadata such as creation and modification dates are also recorded, ensuring that SALN entries are automatically deleted after five days.

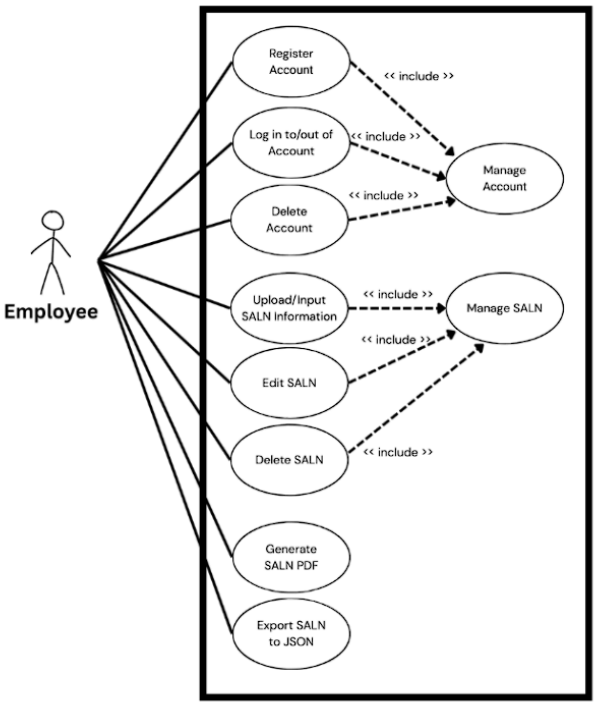
Interaction Model

Actors and Goals

The initiating actors of the system are the employees who need to fill up their SALN form. Their goal is to accomplish their SALN form, save it to their account, and export it to a printable format via PDF or a machine-readable format via JSON.

The participating actors of the system are the browser and DigitalOcean, and administrators. The browser's role is to support offline mode in the app, and to store user data via built-in APIs localStorage, cacheStorage, and IndexedDB. The role of DigitalOcean is to act as the cloud server to host the web server and the remote database.

Use Cases



The main user of our app will be an employee who needs to fill in their SALN.

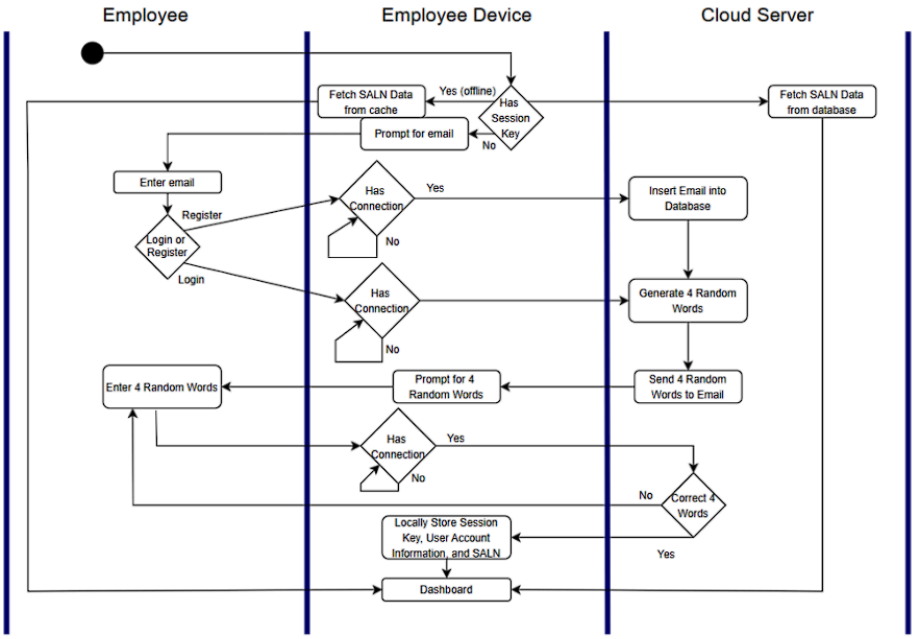
The use cases can be divided into account management, SALN management, and SALN exportation.

Use cases related to account management include registration, logging in to, and logging out of your account. These processes will be secured using a One-Time Password two-factor authentication sent via email.

Use cases related to SALN management include generating a new SALN form through either filling it up in the app or uploading a JSON. Once it's finished, the user may either edit the form or delete it if they so desire.

Lastly, the app comes with a feature to export the completed SALN to either PDF format or JSON format.

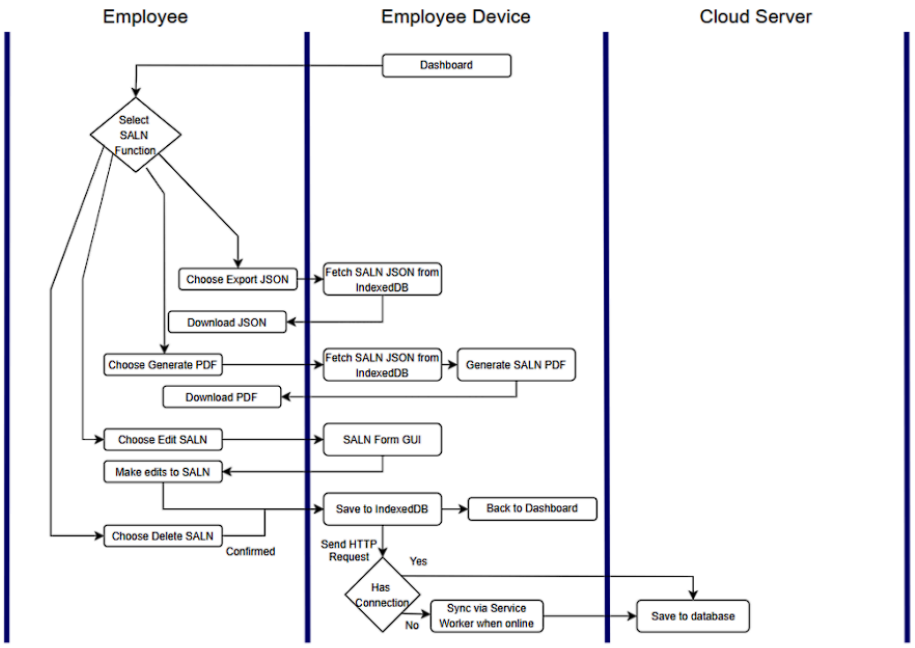
System Sequence Diagrams



The first part of the swimlane diagram of the whole system is the login. The SALN app first checks if the user has a session key stored in IndexedDB. If so, he will immediately be logged in and be sent to the dashboard. If he was automatically logged in offline, he will use the latest SALN data that was cached. If he was automatically logged in online, he will use SALN data from the remote database; This would also allow him to sync his SALN data with the latest SALN data from the remote database.

If the user has no session key in his browser's IndexedDB, that means that it may have expired due to long inactivity (a week), or this is the user's first time on the app. The app will prompt the user for an email. If the email is not yet in the database, then the system will make a new account for it via the registration path. Otherwise, the system will just let the user log into the account associated with that email.

Once the web server has received the email, if the user's email is not yet registered in the database, then the web server will insert it into the database, then it will generate four random words for authentication. If the user's email was already registered, then the web server would go straight to generating the four random words. The web server would then send those words to the email and the app will prompt the user for those four words. Once the user enters four words, the web server will cross check those words. Should those words end up verified, the browser will then store data like session key, user account, and current SALN information locally via built-in browser APIs. Otherwise, the prompt for the four words would repeat. Then, the user will finally be sent to the dashboard.

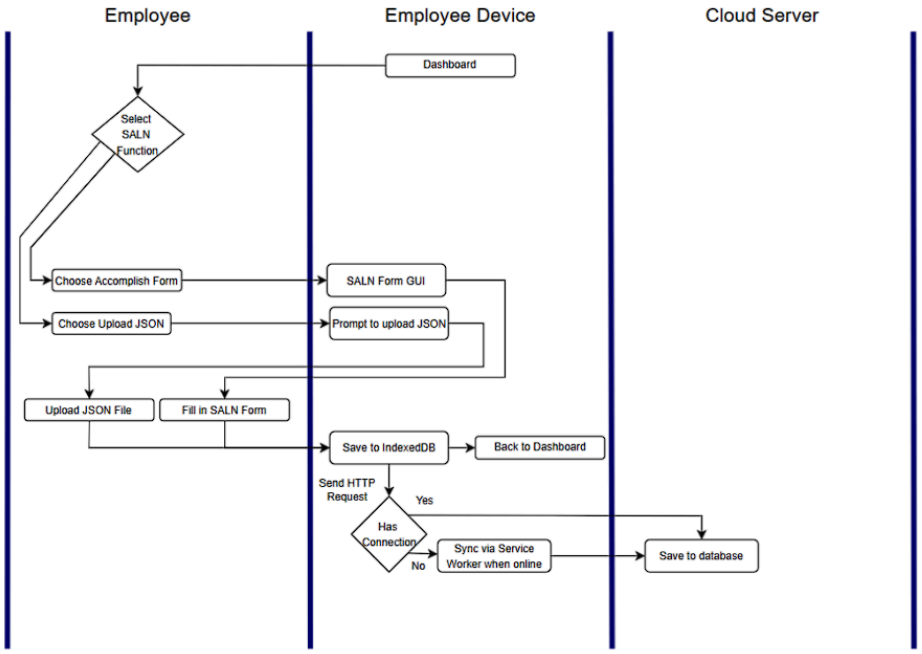


The second swimlane diagram shows use cases in which we are exporting or modifying SALN data from the app. For every SALN form that the user has, he has the option to download the SALN data in JSON format, to generate a PDF version of the SALN form, to edit that SALN form, and to delete that SALN form.

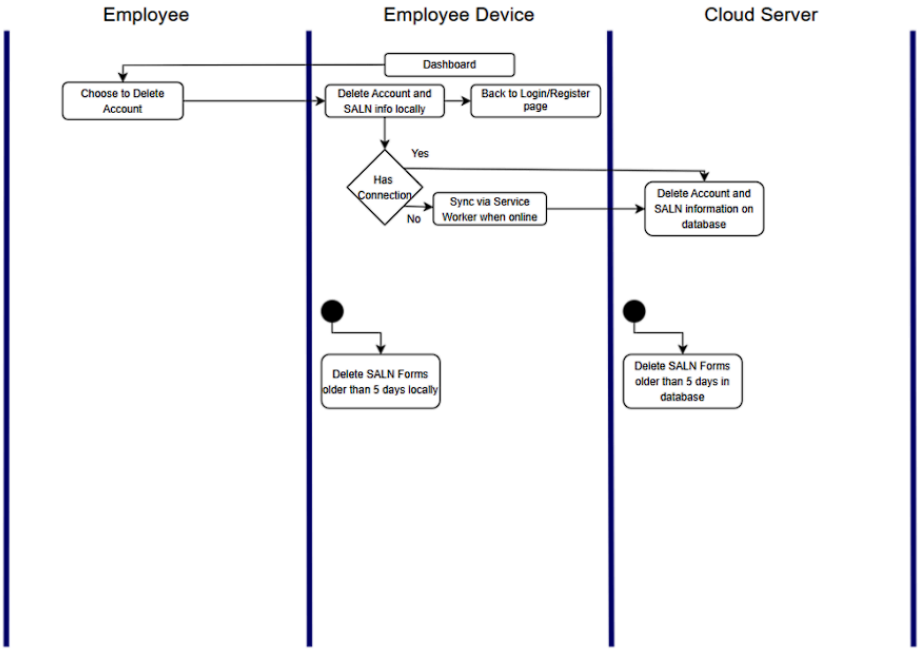
Upon looking at the Export JSON and Generate PDF options, fetching the SALN data is done from IndexedDB. This means that exporting the user's SALN data could be done completely offline.

Upon choosing to edit the SALN form, the user will be sent to the SALN Form GUI. There, he could edit his SALN form data. After that, the app would save those locally in IndexedDB. This allows the user to have an offline update. Then the user would go back to the dashboard. Additionally, the app would send an HTTP request to update the database with the new SALN data. If the user is offline, the Service Worker would save the HTTP Request to IndexedDB and sync it to the database once the user is back online.

Upon choosing to delete the SALN form, the app would ask the user for confirmation. Once the user gives confirmation, it will go through a similar process as the edit function. The local data gets updated, then an HTTP Request is sent to the web server to update the remote database. If the user is offline, then the HTTP Request gets stored into IndexedDB and gets synced once the user is back online via the Service Worker.



The third swimlane diagram contains SALN functions about uploading new SALN data. The user has the option to accomplish the SALN form digitally, or to upload a JSON file containing SALN data. Upon clicking either option, the app will send the user to the appropriate page that will prompt the user for data input. Once the user finishes inputting data, the progressive web app will make a local copy by saving it to IndexedDB and then send the user back to the dashboard. Similar to the previous functions, the Service Worker will also intercept the HTTP request for adding this new data into the remote database.

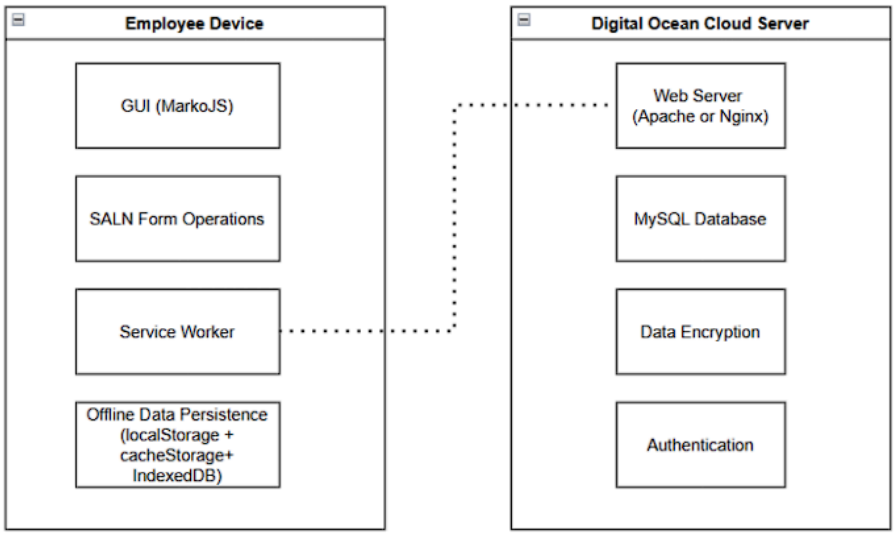


This last swimlane diagram is about data deletion. Aside from the SALN functions in the previous swimlane diagrams, the user will also have the option to delete his account. Doing so has a similar workflow compared to the previous functions. Account information and the SALN data attached to that account will be deleted locally, and then the app will send the user back to the login/registration

page. Furthermore, the Service Worker will intercept the HTTP Request for this deletion in the remote database.

Lastly, though this is not a function that could be done by the user, it is still an important function. The app will automatically delete SALN Forms that are older than 5 days, locally and remotely.

Configuration Model



As a Progressive Web Application, the app could be installed onto the browser's cache, allowing for use whilst online or offline. This installation would include the GUI of the app, SALN Form Operations, Service Worker code, and Offline Data Persistence code. This means that the user should be able to access the GUI and also make use of the SALN Form operations regardless of whether he is online or offline.

The Service Worker code will make use of the Service Worker API, which is provided by the user's browser. Basically, this module works on the logic for syncing the data for offline and online use. Additionally, the offline data persistence module will make use of built-in browser APIs such as localStorage, cacheStorage, and IndexedDB. localStorage will be used to store user account information, like email and user ID. cacheStorage would be used to store the app's assets such as HTML, CSS, and JS files for the frontend, and also the SALN template. IndexedDB acts as a local database, which could store the user's SALN entry, API requests, and session and refresh tokens.

The system will make use of a cloud server by Digital Ocean. For the configuration of the cloud server:

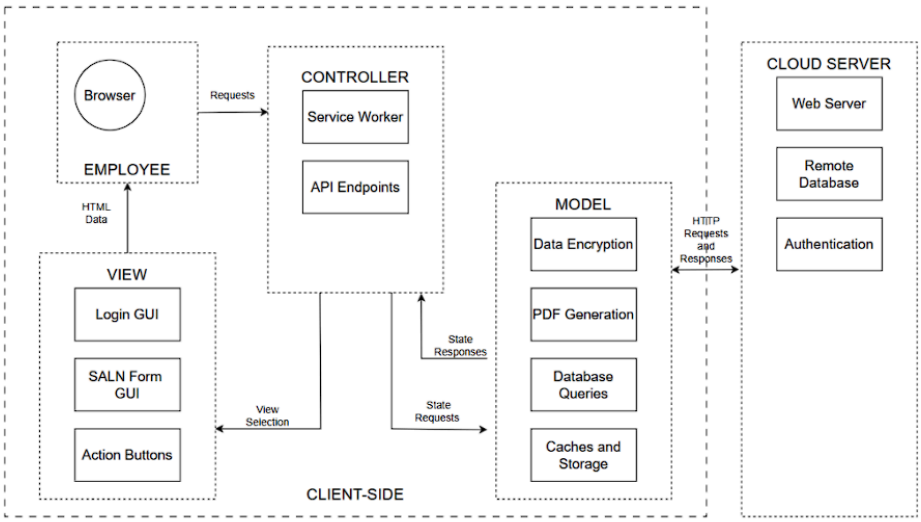
- OS Image: Ubuntu 24.04 (LTS) x64
- Disk Type: SSD
- 1 GB / 1 CPU
- 25 GB SSD Disk
- 1000 GB Transfer

The cloud server would make use of Apache acting as a web server receiving HTTP Requests from clients. It would also make use of MySQL for Database management. Data Encryption would be done using the SubtleCrypto interface of the WebCrypto API. Additionally, authentication logic for log-in and registration will be done on Laravel. A session token could then be saved, and made persistent, in the user's browser's localStorage.

WEB APP ARCHITECTURE AND DESIGN

Architecture Design

Architectural Style



The architectural style that this progressive web application will use is the Model-View-Controller (MVC) Architecture. In the View, we have modules on how to present our data and actions in the Employee's Browser, in the form of HTML data. These include modules on how to present the Login GUI, the SALN Form GUI, and the action buttons for the current app state.

The Controller contains API Endpoints that the Employee could use to interact with the View and the Model. In a sense, it is the endpoint on how the Employee could manage his data through making requests. Additionally, the Controller has the Service Worker, which will determine if the app will be using offline data stored locally or online data stored remotely. The Controller also has the ability to select what the View will be presenting based on the current states of the application in terms of authentication and SALN accomplishment.

The Model contains the backend functionality of the app. It has the logic of determining whether the Employee is authenticated. It also contains methods of fetching the employee's SALN data, either from the remote database or the local cache. The model also encapsulates request handling logic, which includes Database Queries, Queries to the local Cache and Storage, Data Encryption, and PDF Generation. It also has a connection to the external Cloud Server, which encapsulates the Web Server, Remote Database, and Authentication.

It is also worth noting that in order to have persistent offline functionality, everything encapsulated by the model, view, and controller is all on the client side, meaning the code for these would be cached.

Addressing Technical Attributes

Usability. The architecture makes use of GUIs from the View to be displayed on the Employee Browser to interact with functions from the Model in an abstracted way, such that it is understandable what the result of an input would be. Additionally, since this is a progressive web app, responsiveness will be handled by the View component.

Functionality. The Controller will determine what pages the user is allowed to navigate into based on the state of the Model. These allowed navigation are then sent to the View, and then the View would show the necessary GUI elements on the Employee Browser for those navigations. The Model is in charge of processing data from user inputs by storing them locally and in the remote database.

Reliability. To avoid errors related to fetching and updating data in the remote database, the Controller has a Service Worker to intercept requests just in case a connection is unable to be established. For user inputs, the Controller is in charge of user input validation before the data could enter the model to manipulate the database. Additionally, with the MVC Architecture, the practice of separations of concerns is in play. So an error in one of the components shouldn't affect the other components.

Efficiency. Since this is a Progressive Web App, files for the GUI are already stored in cache. This means that the GUI is already available offline and wouldn't need to be fetched from a remote server every time the app is used. Additionally, some other functionality is also kept locally, such as PDF generation. Also, SALN operations have their offline counterparts since the Service Worker in the Controller intercepts requests and then just syncs data on a separate thread.

Maintainability. The separation of concerns via the MVC Architecture allows us to maintain different components separately. If we want to modify backend functionality, then we would just need to edit the Model. If we want to modify how the GUI looks, then we would just need to edit the View.

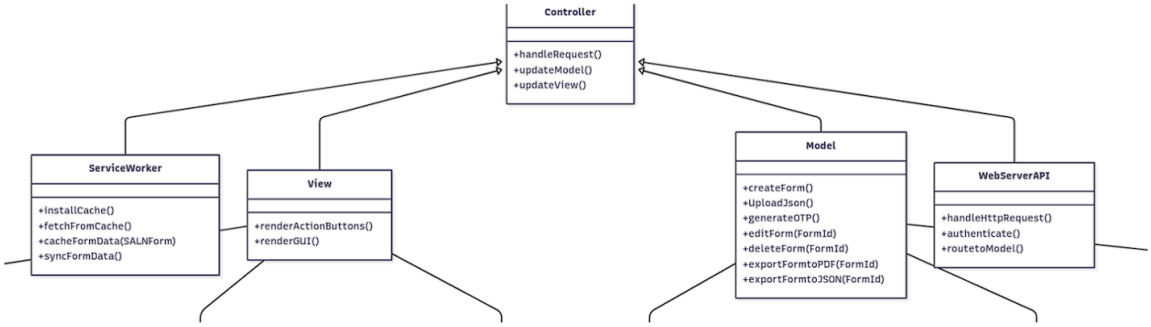
Architectural Decisions

Design Issue	SALN Progressive Web App must have offline functionality. Additionally, it must sync remote SALN data with the local SALN data once back online.
Resolution	Have a Service Worker intercept Requests before they are sent to the Web Server. This way, if the Employee is offline, the Service Worker will store this request and then fetch it once back online. The progressive web app would also maintain a local version of the database via IndexedDB. This way, even in offline mode, the Employee still has access to their latest SALN data. Offline functionality for PDF generation and data encryption would be supported by fully offline dependencies.
Category	Integration
Assumptions	The employee must have his latest SALN data available offline. The Employee must be able to generate PDF offline.
Constraints	Service Workers, IndexedDB, cacheStorage, and localStorage are available built-in browser tools.
Alternatives	Make a native app version alongside the website. This was rejected because the effort needed to create both native app and website would be more. Additionally, the project client requested that no binary be installed for offline functionality.
Argument	From a developer perspective, a progressive web app would be simpler for data syncing because of built-in browser tools like IndexedDB, cacheStorage, and Service Worker API. From a user perspective, this allows the user to perform functions much quicker since everything would just be in cache. The online Requests would be performed on a separate thread.
Implications	Requests would be intercepted by a Service Worker. Database updates would be reflected in local IndexedDB. Code for needed offline functionality would be cached locally.
Related Decisions	None.
Related Concerns	None.

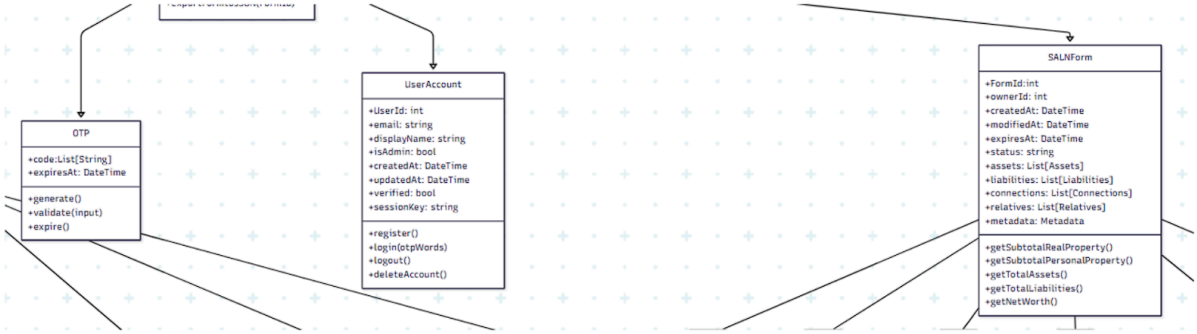
Work Products	Controller: particularly the Service Worker. Model: it contains local cache and storage.
Notes	None.

Content (Data) Design

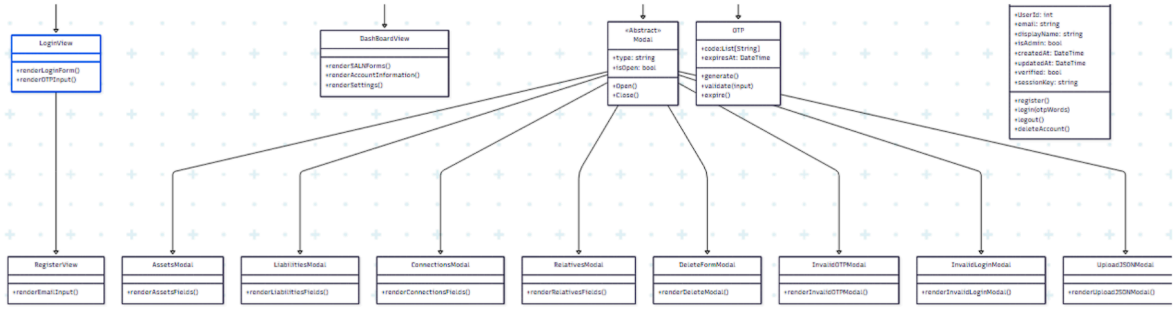
Class Diagrams



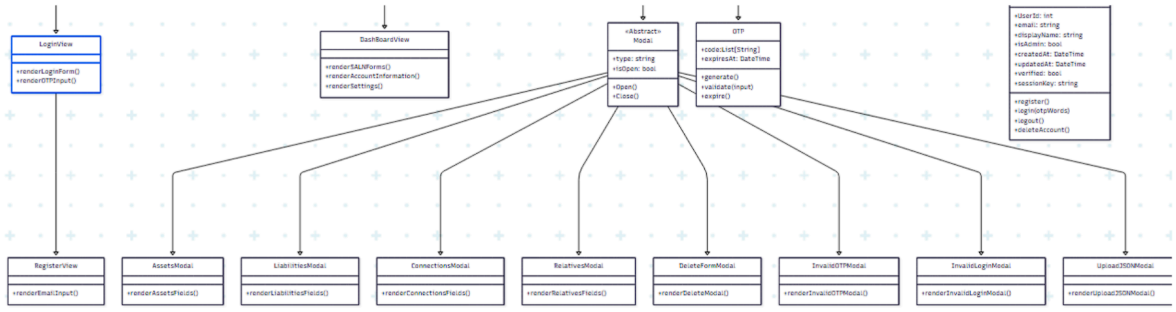
The UML class diagram of the SALN Management System is built on the **Model-View-Controller (MVC) framework**. The **Controller** coordinates requests, updates the Model, and refreshes the View. The **Model** encapsulates business logic, handling SALN forms, data exports, and authentication through OTP generation. The **View** renders the interface (the GUI), while the **WebServerAPI** supports web requests and routing. The **ServiceWorker** complements this by enabling caching, synchronization, and offline access.



Under the Model, in which we handle the state and the data of the Webapp, we have the **OTP**, the **SALN Form**, and the **User Account**. The OTP class contains the OTP itself and its needed details. The SALNForm class encapsulates its attributes, such as assets, liabilities, connections, relatives, and metadata, while also providing the operations to compute subtotals and net worth. The UserAccount class manages registration, login, session handling, and account verification, ensuring that each form is tied securely to its owner.

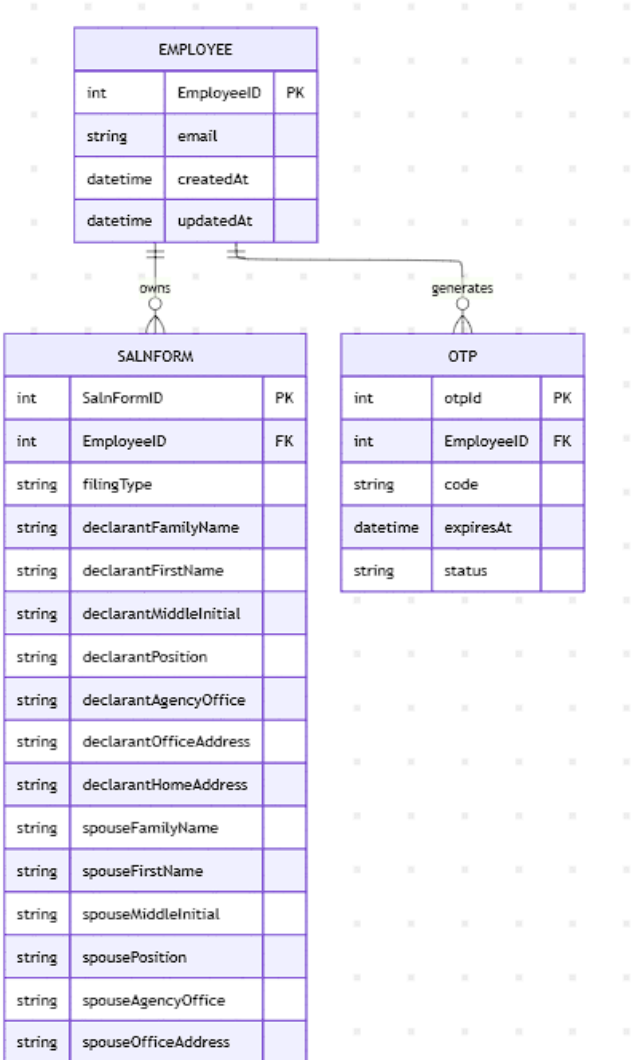


For the SALN form class, here are its main details separated within classes. The **PersonalInformation** class has a **Child** subclass since a list of **Child** is needed in the **PersonalInformation** class The abstract class **Asset** makes the implementation of **RealProperty** and **PersonalProperty** class easier (since they are related and have the same attributes). The **Metadata** class here is important for the system to know when to delete the file, and the owner of the SALN.



Under the **View** Class, we have the **LoginView** subclass, **DashboardView** subclass, and the **Modal** abstract subclass. The LoginView is where the User will start first when visiting the WebApp, and if the user has no account then he will need the RegisterView. The Dashboard view is essentially the Dashboard for the users, where they can access their account info, settings, and their SALN forms. For the Modal superclass, these are where the essential popups or fields for the SALN forms are rendered (AssetsModal, LiabiltiesModal, DeleteFormModal).

Entity-Relationship Diagrams



EMPLOYEE is a table that contains the information stored in each employee account. This includes the following:

- EmployeeID - an integer serving as the unique identifier for each employee account. This serves as the primary key for this table.
- email - a string denoting the email address inputted by the employee during registration.
- createdAt - a datetime indicating the date and time the account was created.
- updatedAt - a datetime indicating the last time the employee made edits to an SALN form.

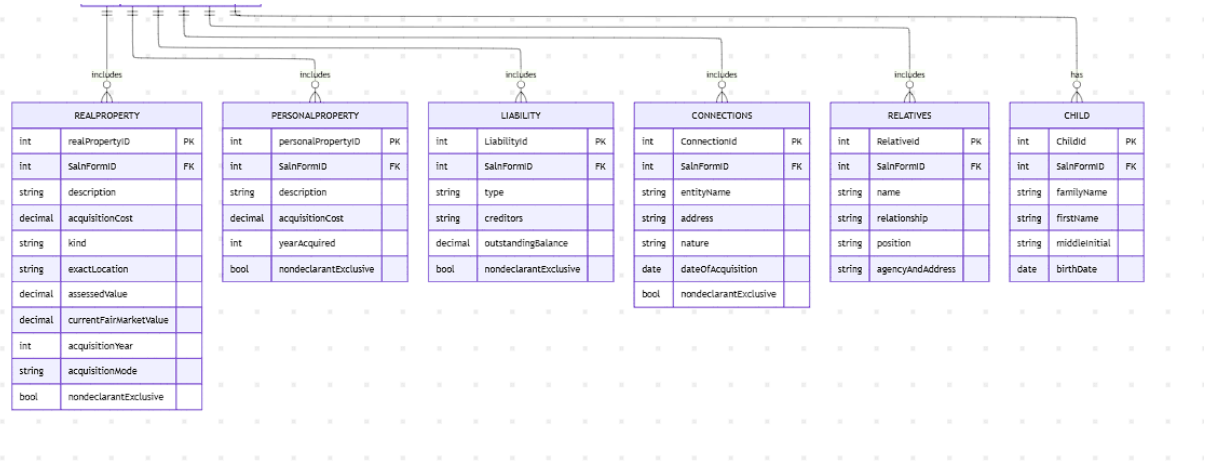
OTP is a table that contains the OTPs generated by the app. Every week, expired and used OTPs are deleted from the database. The attributes of this table include the following:

- otpId - an integer serving as the unique identifier for each OTP. This serves as the primary key for this table.
- EmployeeID - an integer that identifies the employee who needs to enter this OTP. This is a foreign key that relates the OTP to the EMPLOYEE table.
- code - a string that denotes the passcode to be entered by the employee. This consists of four words concatenated.
- expiresAt - a datetime indicating the date and time that the OTP expires.
- status - a string indicating the status of the OTP.

SALNFORM is a table that contains the SalnFormID used to differentiate between multiple SALN records and the EmployeeID used to identify the author of the SALN. This table also directly stores the personal information of the declarant and spouse, such as filing type, names, positions, agency information, and addresses. The other main contents of the SALN are stored in the tables below it, namely REALPROPERTY, PERSONALPROPERTY, LIABILITY, CONNECTIONS, RELATIVES, and CHILD. The METADATA table has been removed, since the information it stored is no longer separated into its own table.

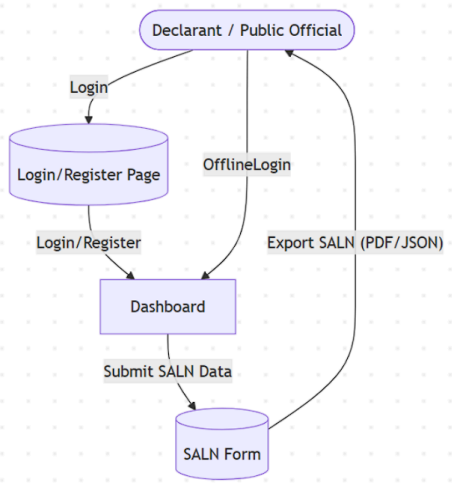
REALPROPERTY, PERSONALPROPERTY, LIABILITY, and CONNECTIONS are all child tables of SALNFORM. Each of these tables uses SalnFormID as a foreign key. They also include a nondeclarantExclusive field to indicate whether the item belongs exclusively to the declarant. The REALPROPERTY table uses realPropertyID as its primary key, while the PERSONALPROPERTY table uses personalPropertyID as its primary key.

RELATIVES and CHILD are also linked to SALNFORM through SalnFormID. These tables store information about the declarant's relatives and children, respectively.



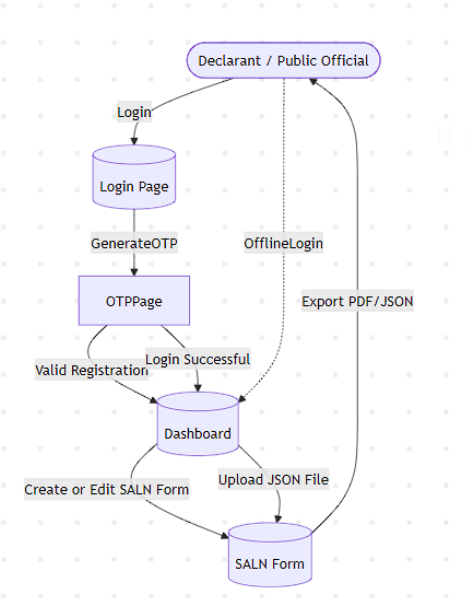
Data Flow Diagrams

LVL 0 DATA FLOW DIAGRAM



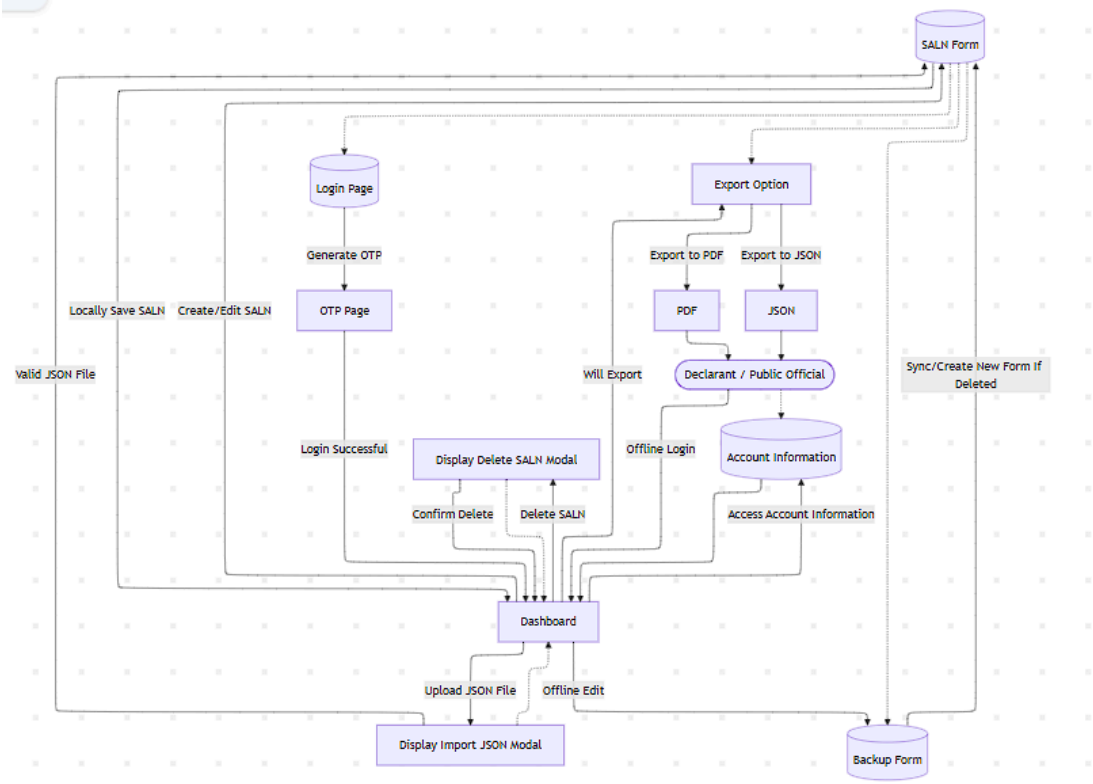
The Level 0 Data Flow Diagram exhibits the main functionality of the system. The User will just Login(Online/Offline) or Register(If the user has no account yet), Visit the dashboard to Submit/Fillup SALN Data, then it can be exported as PDF/JSON by the user.

LVL 1 DATA FLOW DIAGRAM



For the Level 1 Data Flow Diagram, the Login/Registration process is just expanded to show the step by step process of logging in the system. We can see that we need an OTP when we login to the system (Online) to validate the user. Then if the user sent the correct OTP, he/she can know access the Dashboard to edit or upload a JSON file for their SALNform.

LVL 2 DATA FLOW DIAGRAM

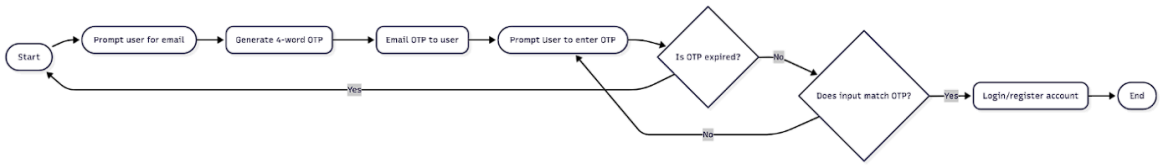


For the level 2 Diagram, we just added the other information we can get from the Dashboard, and how the system works when Creating/Editing SALNforms offline or online. We can see from the data flow diagram above that using the Dashboard, you can access your account information, settings, deleteSALNforms, create/edit SALNforms, and upload a JSONfile that contains the SALNform data. For the Offline editing of the SALN form, we implemented a policy that if the SALN data is deleted online by another device, the cached offline SALN data will be moved to a new SALN Form with a different FormId.

Component-Level Design

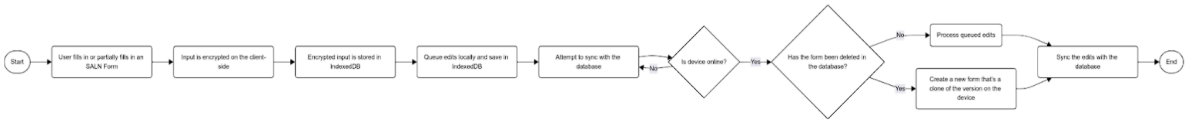
Algorithms

a. UML Activity Diagram for OTP Generation and Emailing



At the login page, the user will be prompted to enter an email address. Once they have done so, an OTP will be generated by the backend by doing the following: generating a random number, then using it to select four random words from the EFF's Long Wordlist. The OTP will then consist of the OTP along with a timestamp for the expiry date. An email will then be sent to the user containing the OTP using the built-in Laravel Mail API. The app will then prompt the user to enter the OTP they received. Once the user inputs, the app will check the OTP timestamp to check if it has expired or already been used. If so, it would create a pop-up, then send the user back to the login page. If the OTP has not expired, then the app would check if the user's input matches the OTP. If it does not match, the user will be prompted again. If it does, then the user will successfully be registered or logged in to their account.

b. UML Activity Diagram for CRUD



Whenever a user makes an edit to an SALN form, the updated information gets encrypted, then stored in IndexedDB. Next, the edits are stored in a queue. The next time the device goes online, the app will attempt to synchronize with the database. On the backend, the incoming edits will be processed one by one using the queue. For each edit, if the corresponding form exists in the database, then the backend will save the edits then sync with devices logged into the account. If not, this indicates that the form was deleted through either auto-deletion or deletion from another device. A new form will then be created with the contents from the local version copied to it.

c. UML Activity Diagram for PDF Generation

When the user requests to generate the PDF for a form, the app first retrieves the JSON containing the form inputs from IndexedDB and checks if all the required entries of the form, other than the signature, are filled. If not, a pop-up will show up saying the form has not been completed, then the user will be redirected to the dashboard. If the form input is valid, then the values in the JSON will be filled into the inputs of the SALN PDF template stored in IndexedDB. Lastly, the PDF will be rendered, then downloaded to the user's computer.

Data Structures

All of the information the user inputs in the forms will be stored in JSONs. This allows for the storage of information in the cache for offline functionality and the exchange of information with the database for CRUD.

The app will make use of a queue to store and process form edits. Because queues are First In, First Out, edits will be processed in the order they are done chronologically, with the timestamp deciding which edits will be saved. As a result, conflicting edits and histories between devices logged into the same account and the database can be resolved.

Interface Elements/Usability Design

When the employee logs into the app, he will be brought to the login interface. There, he will be prompted for his email address. Additionally, if the employee does not have an account yet, he could click the register link at the bottom of the modal. The registration process will be similar to that of the logging in when it comes to the interface and necessary user input.

Figure 13: Login Page of the App

After entering the email, the web server will send said email four random words for OTP purposes. The employee will then be prompted to enter those four words.

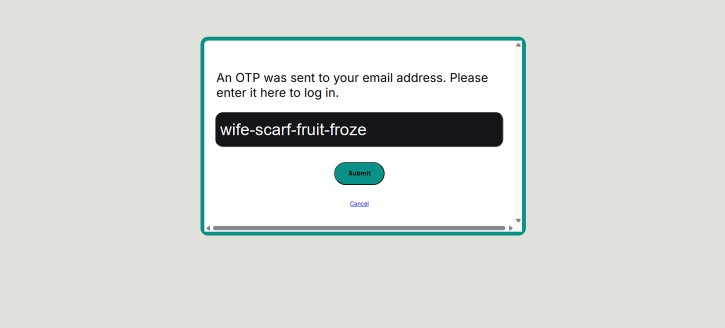


Figure 14: OTP Page of the App

Once the employee has entered the correct four words, he will be logged into the app and redirected to the dashboard page. Once on the dashboard, the SALN data of the employee will be displayed alongside with other action buttons. On the top right corner, there are buttons for general actions that could be performed. In general, an employee could choose to accomplish a new SALN form, to upload SALN data in JSON format, or to delete his account. Additionally, a list of the employee's currently available SALN forms will be displayed. Each displayed SALN form has four buttons for possible actions: to export as PDF, to export as JSON, to edit the data, and to delete the data.

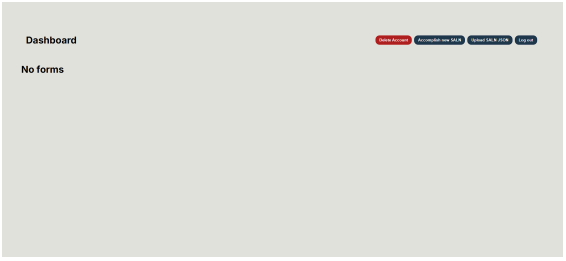


Figure 15: Empty Dashboard

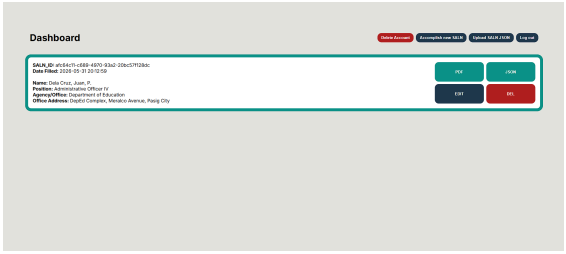


Figure 16: Dashboard with One Entry

If the employee clicks the Accomplish SALN button, he will be sent to the digitalized SALN form to input SALN data. The fields being prompted mirror that of the physical SALN form. For example, Figure 17 shows the first page of the digital form taking in some personal information, which reflects the first few fields of the physical SALN form. Then, at the bottom of the page, there are buttons for navigating between SALN form pages.

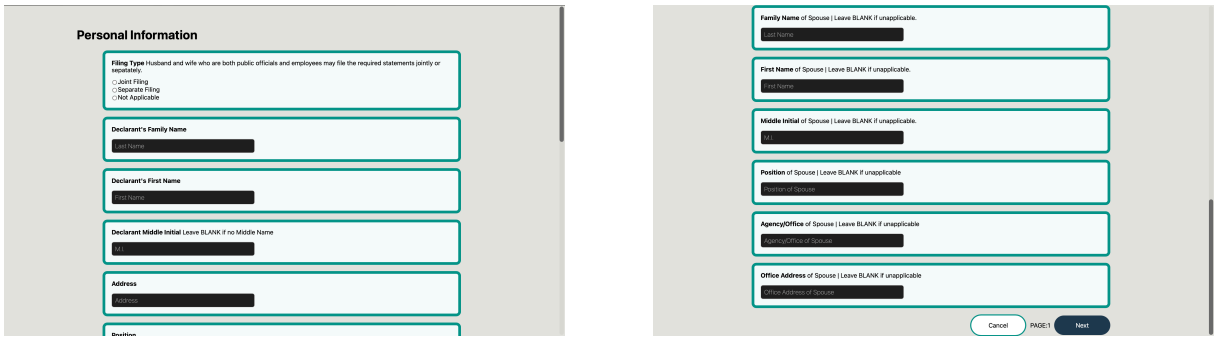


Figure 17: Personal Information Page of the Digitized SALN Form

It is also worth noting that on the physical SALN form, there are tables for assets and liabilities. The digitalized SALN form displays asset fields in a table format. All tables from the physical SALN will also share this GUI layout. On the top right, there is an action button to add another asset. For each asset, there is an option to edit the asset and then there is an option to delete the asset.

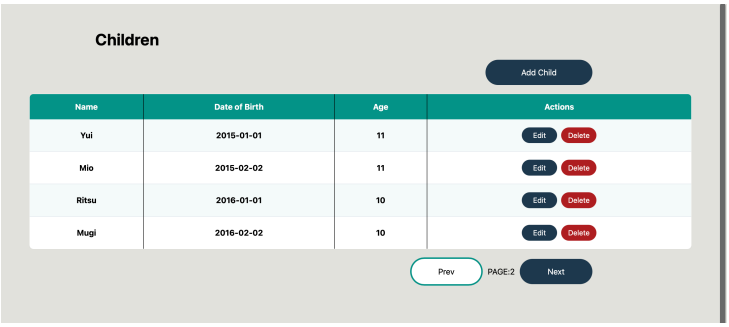


Figure 18: Unmarried Children Page of Digitized SALN.

When adding an asset, a modal prompting for asset fields will appear at the center of the screen.

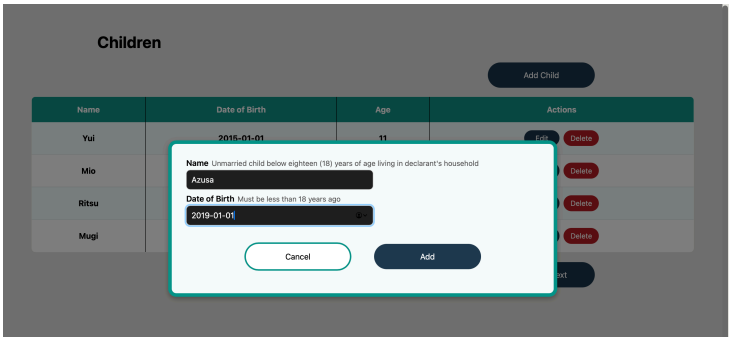


Figure 19: Modal for Adding Entry to Unmarried Children Table in Digitized SALN

Going back to the dashboard, there was also an option to upload a JSON file containing SALN form data. Clicking that navigation button will send the employee to a page prompting them to upload a JSON file. Additionally, there is a Help button at the top right corner of the modal. This button will bring the employee over to documentation of the JSON format for SALN form data.

Figure 20: Upload JSON Page

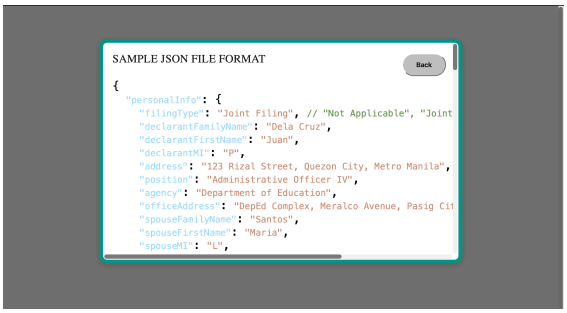


Figure 21: Upload JSON Help Page

It is worth noting that the app also deals with error handling. This is done by having a modal pop up. For example, a user tries to generate a PDF version of his SALN form despite said SALN form being incomplete.

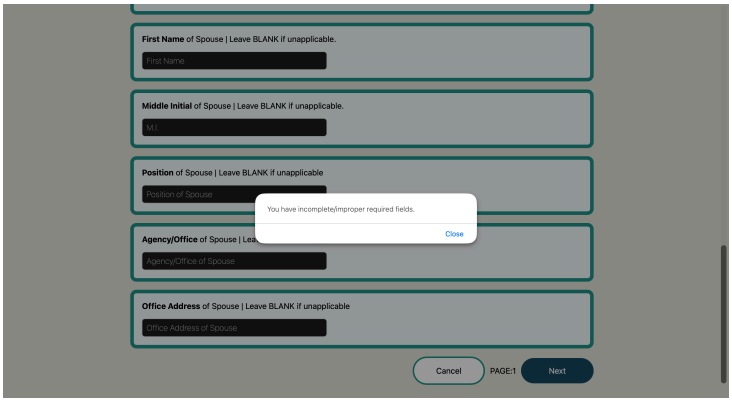


Figure 22: Error Modal

The app also has user error prevention. For example, upon clicking a Delete button, the app will have an alert pop up asking for confirmation. For instance, when clicking the Delete button for a SALN form, a prompt will show up asking if the user wants to delete the SALN form along with the SALN form's distinguishing fields.

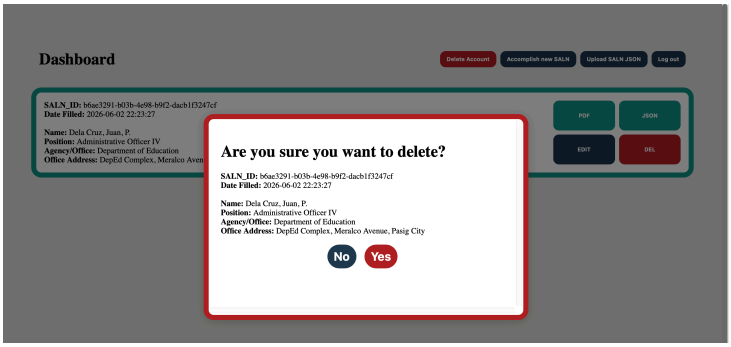


Figure 23: Confirmation Modal for Deleting a SALN Form Entry.

Additionally, since there are no complex animations in the GUI, rendering the graphics should be quick. Additionally, since the data to be used is coming from local cache, there won't be much overhead in fetching form data to be displayed compared to fetching form data from the remote database.

Interface Design - Menus and Links

Navigation mechanisms come in the form of buttons with an action function attached to them as attributes. Their layouts could be seen in the storyboard images above.

Starting off with the login interface asking for the user's email, there would be a button bringing them to the prompt for four random words. Additionally, there would also be a link sending the user to the email registration page. After successfully entering the four random words, the user would be sent to the dashboard upon clicking the Login button. This behavior could also be seen with the registration process. The only difference would be that there would now be a link sending the user to the login page.

In the dashboard, there are three general action buttons. For each SALN form, there are four action buttons that will affect only that SALN form.

Going into the digital SALN form, navigation buttons could be seen in the bottom right corner. The goal is to go forward or backward in the sequence of form pages. Assets and Liabilities pages have similar patterns. In general, assets would have an Add Asset button making a modal pop up for Asset input. For each asset, there would be two action buttons affecting only that asset.

Aesthetic Design - Layout and Graphics
This is the color scheme to be used:

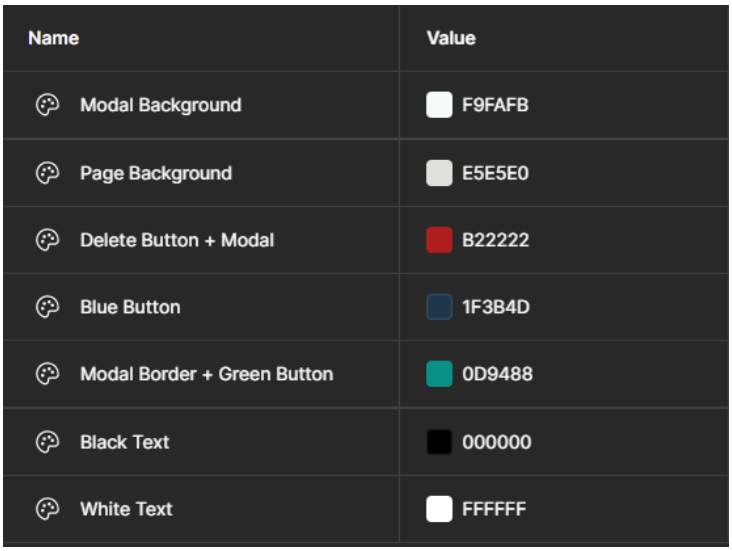


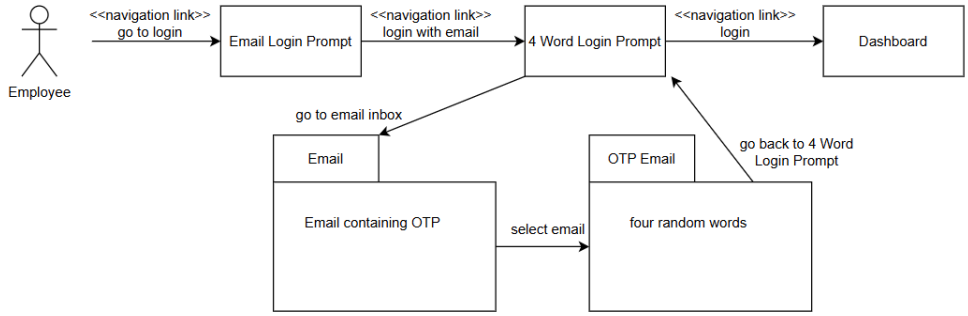
Figure 24: Color Scheme of the App.

Height will just be flexible depending on how much content is inside these elements. For relative width sizes:

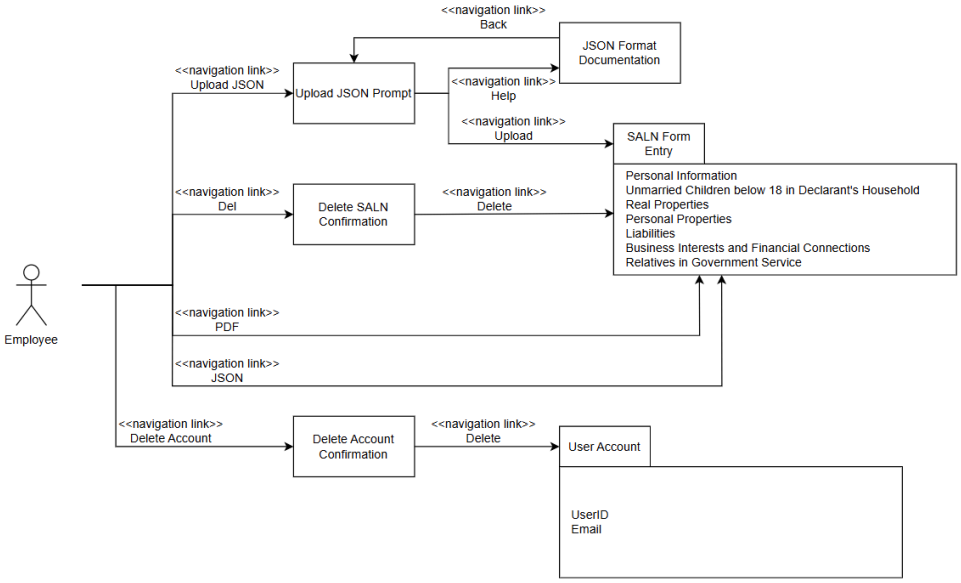
ELEMENT	SCREEN WIDTH %
Login Modal	25%
General Action Button	14%
List Modal	80%
Entity Specific Action Button	6%
General Popup Modal	45%
Asset/Liabilities Popup Modal	55%

Navigation Design

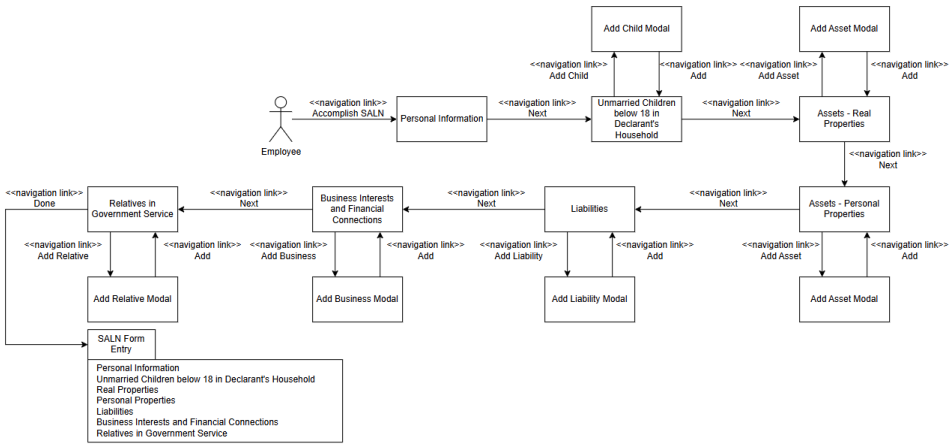
One common scenario is the login process. Registration also mirrors this navigation scheme. Upon opening the app, the user is sent to the login page prompting for the user's email. Then, the user inputs his email. Let's say it is around 20 characters long. Then he clicks the Login With Email button, sending him to a 4-Word Login Prompt. He then goes to his email inbox, then opens the OTP email that contains the four randomly generated words. Then he goes back to the prompt and then enters the 4-Word OTP. Let's say it is around 20 characters long. Lastly he clicks the Login button and gets sent to the dashboard. In this case, there are 6 mouse clicks and around 40 keystrokes, 6/46 of which are used for navigation while 40/46 are used for data entry.



Another common scenario will be accomplishing a SALN form. Consider this scenario where the employee has no spouse, no children, no family working in the government, and only one real property. We will also start analysis from the dashboard. The user clicks the Accomplish SALN button, which will send him to the first page of the digital SALN Form to enter personal information. Suppose that takes around 145 characters given the specifications of the scenario. Then, he will click Next which will send him to the page asking for unmarried children. Since he has none, he will just click Next and get sent to the real properties page. In this, he has one real property. So, he has to click the Add Asset button to put in information about that real property. Suppose this real property takes around 93 characters for all its fields. Then he would click Add once finished with the real property, and then Next to go to the next page of the SALN form. Then for the succeeding pages, since he has no entries for those, he will just click the Next button for them until he is Done. There are 238 keystrokes and there are 10 mouse clicks, 238 out of 148 are used for data entry, and 10 are used for navigation. It is also worth noting that editing a SALN form would have a similar navigation scheme to this with potentially less keystrokes since the user would be changing entries of less fields.



The next functions of the dashboard are quite straightforward. Exporting only needs one navigational click. Deleting an account or a SALN form will need two navigational clicks, one for calling the function and one for confirmation. Uploading the JSON prompt needs two navigational clicks, one going into the prompt and one confirming the upload.



III. Software Maintenance Plan

System Description

Points of Contact

SALN Progressive Web App Development Team
- AGCAOILI, Lucas Miguel V. - lvagcaoili@up.edu.ph
- GUIANG, Miguel - maguiang@up.edu.ph
- PEÑAFLORIDA, Aeron Dann - appenaflorida@up.edu.ph

Security

OTP Verification

The login and registration processes for the client app make use of an OTP for user verification. They work as follows:
1. The user enters their email to log into/register their account.
2. The user receives a 4-word OTP via email to verify their account.
3. The user enters the OTP, giving them access to the app.

UUID and Email Identification

To uniquely identify users, the app makes use of a universally unique identifier (UUID) and email as the primary key for database operations.

Data Encryption in the Database

Account information and SALN data are encrypted using the built-in encryption functions in the Laravel framework of PHP.

Auto-Deleting Old Data

To ensure that the server does not contain sensitive information for long periods of time, SALNs which have not been updated in 5 days get automatically deleted from the database.

.env file

To configure the server, an `.env` file is needed. These file contains sensitive information, so they are not to be shared or pushed to any public repositories. These could be gotten from the developers stated in the Points of Contact section.

Computer Hardware

The project only requires a computer to run. While in development, the server is hosted on a computer with the `.env` file.

Eventually, the server will be done on the cloud using Digital Ocean. It will use the Linux operating system.

Support Software

To test the project properly, the dependencies for both the client and server side are listed down below:

- Client-side
  - marko/run (0.8.1) - development server for Marko projects
  - marko (6.0.82) - UI framework used to build the app's pages/components
  - pdf-lib (1.17.1) — used for creating and manipulating PDF files (e.g., generating SALN forms)
  - uuid (13.0.0) — generates unique IDs for records and internal identifiers
- Server-side
  - PHP (8.3.6) - main server-side programming language used to run the backend logic

- Laravel (12.33.0) - backend web framework used for routing, controllers, validation, database access, and API endpoints
- Laravel Sanctum (4.2.0) - provides a featherweight authentication system for SPAs and simple APIs.
- Composer (2.7.1) - PHP dependency manager used to install and update backend libraries/packages

Personnel

This section identifies the personnel roles required to maintain SALN Progressive Web App and the skills needed to perform routine maintenance, troubleshooting, and updates.

- Maintenance Roles and Responsibilities
  - Lead Developer
    - Acts as the main contact for the client
    - Decides what gets fixed first (urgent issues vs. nice-to-haves)
    - Keeps the team organized and makes sure releases actually happen
  - Backend Developer (must know PHP and Laravel)
    - Maintains the server-side features and API endpoints
    - Fixes issues related to OTP generation/verification and email sending
    - Handles database migrations and security updates
  - Frontend Developer (must know Marko, CSS, HTML, and JavaScript)
    - Fixes UI issues (forms, validation, layout, responsiveness)
    - Makes sure the PWA still behaves properly across browsers/devices
    - Maintains PDF-related features (generation/export) and client-side flow

Maintenance Procedure

Conventions

In general, the naming procedure used for variables in both the JavaScript (MarkoJS) client and PHP (Laravel) server sides is `camelCase`, with capitalization being used for `Class` names. Field names in JSON messages are also kept on `camelCase`. Additionally, `SCREAMING_SNAKE_CASE` is used for `.env` variables.

In terms of comments, since Javascript, and by extension MarkoJS, is not all that strict with type annotations, it would be helpful to at least comment them out before function definitions. For example:

```
/**
 * Register a new employee
 * @param {string} email
 */
export async function registerEmployee(email)
```

Maintenance Procedure

Upon downloading the GitHub repository, go into the root project directory. In there are two sub-directories: `saln-client` and `saln-server`. We need to set both of them up.

Client Setup

In setting up `saln-client`, all we really need to do is simply install the dependencies using `npm`. `npm` is the package manager of `Node.js`. You could get it from here if you don't have it yet, https://nodejs.org/en/download. Then, install the dependencies:

```
cd saln-client
npm install
```

Then, we could run the developer version of the MarkoJS client in side in `saln-client` directory via

```
npm run dev
```

This allows us to see the app in `localhost:3000`.

Server Setup

In setting up `saln-server`, we need to install MySQL and PHP/Laravel, and then make some changes to the local configuration.

1. Installing MySQL:

```
sudo apt update
sudo apt install mysql-server -y
sudo service mysql start
```

2. Installing PHP:

```
sudo apt update && sudo apt upgrade -y
sudo apt install php php-cli php-mbstring php-xml php-bcmath php-json php-zip php-
curl php-mysql unzip curl git -y
```

3. Installing Composer, PHP's package manager:

```
sudo apt install composer -y
```

4. Installing Laravel:

```
composer global require laravel/installer
echo 'export PATH="$PATH:$HOME/.config/composer/vendor/bin"' >> ~/.bashrc
source ~/.bashrc
composer install
```

5. On MySQL, create the database that Laravel will connect to:

```
CREATE DATABASE saln_app_DB;
CREATE USER 'saln_user'@'localhost' IDENTIFIED BY 'saln_password';
GRANT ALL PRIVILEGES ON saln_app_DB.* TO 'saln_user'@'localhost';
FLUSH PRIVILEGES;
```

6. We also need to configure our Laravel instance to use that newly created database. In `saln-server/.env`:

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=saln_app_DB
DB_USERNAME=saln_user
DB_PASSWORD=saln_password
```

7. We also want to generate a master key to be used for data encryption and create the database tables from the Laravel migrations:

```
php artisan key:generate
php artisan migrate:fresh
```

8. Lastly, we need to set up a cron job for scheduled tasks.

```
crontab -e
```

9. Then on the bottom of the file, put these commands:

```
* * * * * cd /Path/to/saln-app/saln-server && php artisan schedule:run >> /dev/null
2>&1
```

10. Finally, to run the server:

```
cd saln-server
php artisan serve
```

Verification Procedures

When developing on the front-end client, simply running the client `npm run dev` to start the app for visual inspection of the page's elements will do.

Alongside client testing, you would most likely be testing API accesses from the client side. To see all API calls while the Laravel server-side is running, you could look towards the terminal instance running `php artisan serve`.

Additionally, server-side errors could be seen in `saln-server/storage/logs/laravel.log`. This is also where errors coming from MySQL queries will be printed into. For further testing regarding MySQL, you could always log your user into the MySQL server and check the database directly.

Server-side error logs related to the NGINX web server could be found in `/var/log/nginx/error.log`.

Error Conditions

MySQL Errors:

Usually this is caused when the expected payload from client does not match what the Laravel server-side is expecting. This leads to missing fields in the MySQL query. Another potential cause is the other way around: Laravel is not what the client expects. For instance, you changed the schema of some table in the `migrations` folder of Laravel, but forgot to run `php artisan migrate` to update the schema of the database in the MySQL server.

MarkoJS frontend not updating after code changes: The service worker caches the static Javascript files when the app is being served. This means that even if you refresh MarkoJS after making changes to the code, there is a possibility that the service worker would have already cached the previous code, making you use that said previous code. To fix this, delete the browser's cache at `localhost:3000` and unregister the service worker so that the new one could take its place.